

Computer Programming Summer Interest Group

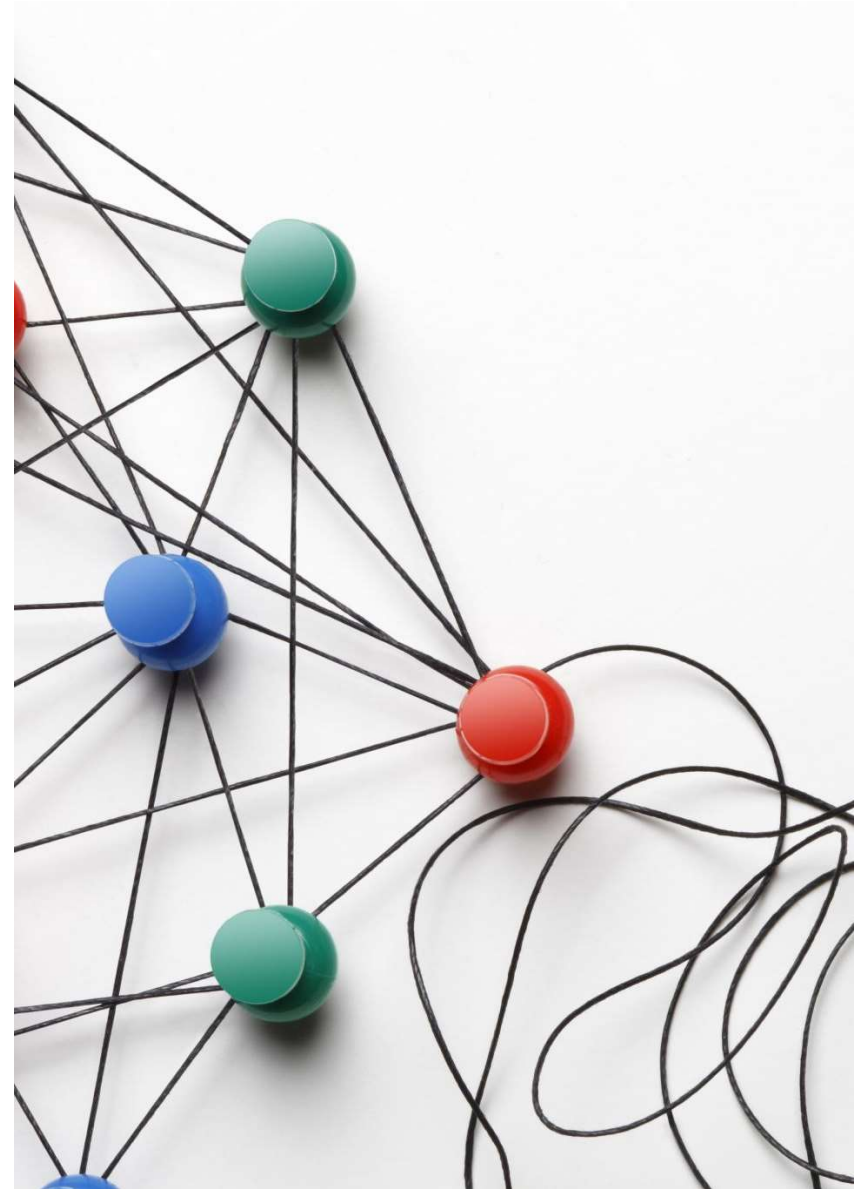
Session 15 / Inheritance

Ephesians 1:11-14

In him we have obtained an inheritance, having been predestined according to the purpose of him who works all things according to the counsel of his will, so that we who were the first to hope in Christ might be to the praise of his glory. In him you also, when you heard the word of truth, the gospel of your salvation, and believed in him, were sealed with the promised Holy Spirit, who is the guarantee of our inheritance until we acquire possession of it, to the praise of his glory.

Objectives

- By the end of this session, you will be able to:
 - Understand the concept of inheritance and why it is useful.
 - Learn how to define a parent (base) class and a child (derived) class.
 - Explore how attributes and methods are inherited from parent to child classes.
 - Understand method overriding and the use of `super()`.
 - Practice creating class hierarchies using inheritance.



Class Inheritance

Recall a class is an association of attributes (data) and methods (functions)

Inheritance is a concept in object-oriented programming

A “child” class may “inherit” the methods and attributes of a “parent” class.

This allows structure and function to be reused to refine the behavior of child classes.

Inheritance Warnings

- If improperly used, inheritance can make a program more complex than needed
 - This leads to tight coupling and low cohesion which makes later modification more difficult
- Apply the [Open / Closed Principle](#) when using inheritance
 - Open for extension – A child class should add attributes and methods to a parent class
 - Closed for modification – A child class should not modify the meaning of attributes or the definition of methods defined in the parent class
- In general, resist the urge to use inheritance, but there are times when it is useful
 - Prefer [Composition over Inheritance](#)

Terminology

Parent class, base class

- A class from which other classes inherit attributes and methods

Child class, sub-class, derived class

- A class which inherits attributes and methods from a parent class

Object, Instance

- A single realization of a class

Instantiate

- To create an instance of a class

Override

- To change the definition of a method between parent and child class instances

Simple Example

- Parent class “Vehicle”
 - Provides an attribute “vehicle_type” and a method “print_type”
- First child class “Automobile”
 - Initializer uses “super” to invoke the parent class __init__ method
 - Extends the parent class by adding the “drive” method
- Second child class “Airplane”
 - Initializer also uses “super” to call parent class __init__ method
 - Adds the “fly” method
- Code on right creates instances and uses the defined methods

```
class Vehicle:  
    def __init__(self, vehicle_type: str):  
        self.vehicle_type = vehicle_type
```

```
    def print_type(self):  
        print(self.vehicle_type)
```

```
class Automobile(Vehicle):  
    def __init__(self):  
        super().__init__("automobile")
```

```
    def drive(self):  
        print("driving")
```

```
class Airplane(Vehicle):  
    def __init__(self):  
        super().__init__("airplane")
```

```
    def fly(self):  
        print("flying")
```

```
car = Automobile()  
plane = Airplane()
```

```
car.print_type()  
car.drive()  
plane.print_type()  
plane.fly()
```


Overriding Methods

- Parent class “Vehicle”
 - Provides a new “move” method
- Child class “Automobile” overrides the parent class “move” method
 - This has the effect of redefining how the move method works when associated with Automobile rather than Vehicle
- Using “super” in the move method
 - Calls the parent class implementation of the method
 - Can be called anywhere inside the child class method.

```
class Vehicle:
    def __init__(self, vehicle_type: str):
        self.vehicle_type = vehicle_type

    def print_type(self):
        print(self.vehicle_type)

    def move(self):
        self.print_type()

class Automobile(Vehicle):
    def __init__(self):
        super().__init__("automobile")

    def move(self):
        super().move()
        print("driving")

car = Automobile()
car.move()

generic = Vehicle("generic")
generic.move()
```


Multiple Inheritance

- It is possible to form a class by inheriting from multiple parent classes
- This is called multiple inheritance
- Example defines an “Omnivore” class by combining a “Vegetarian” and “Carnivore” class
- Notice the `__init__` method in Omnivore must call the `__init__` methods for both parent classes
 - This is done by specifically naming the parent class and using the `__init__` method to prevent confusion about which parent class’s `__init__` method is meant.
- We then can create an instance (raccoon), which is a certain kind of omnivore
- Note this is a contrived example.
- If you should be careful with inheritance, you need to be extra super careful if you are considering using multiple inheritance

```
class Vegetarian:
    def __init__(self, favorite):
        self._favorite_vegetable = favorite

    def favorite_vegetable(self):
        print(f"Favorite vegetable: {self._favorite_vegetable}")

class Carnivore:
    def __init__(self, favorite):
        self._favorite_meat = favorite

    def favorite_meat(self):
        print(f"Favorite meat: {self._favorite_meat}")

class Omnivore(Vegetarian, Carnivore):
    def __init__(self, vegetable, meat):
        Vegetarian.__init__(self, vegetable)
        Carnivore.__init__(self, meat)

    def favorite_foods(self):
        self.favorite_vegetable()
        self.favorite_meat()

raccoon = Omnivore("cucumbers", "snake")
raccoon.favorite_foods()
```



Remember...

Inheritance
needs to be used
carefully.

There are safe
ways to use it,
even multiple
inheritance

I will present
some safe ways
to use it in later
sessions



Activities

- Create a base class Shape with area() and perimeter() methods.
 - Create subclasses Circle and Rectangle that override area()
- Write a base class Employee with attributes name and salary.
 - Create a subclass Manager that adds a department attribute.